

IoT Device Security Assessment

Penetration Testing – Continuous Assessment Project

Target: OWASP IoTGoat v1.0 (192.168.57.128)

Testing Machine: Kali Linux (192.168.57.129)

Date: 13 May 2026

Module: Penetration Testing

For Academic Use Only

1. Introduction

This report is written as part of a Continuous Assessment project for the Penetration Testing module. The task was to assess the security of IoT devices deployed across an organisation, find any weak configurations, and evaluate how much risk those weaknesses create.

To carry out this assessment, I used OWASP IoTGoat, which is a deliberately insecure IoT firmware image designed for learning and testing purposes. The testing was done in a safe, isolated VMware virtual network with no connection to any real systems or the internet. My Kali Linux machine was used as the attacking machine throughout the process.

The goal of this report is to document all the vulnerabilities I found, explain what they mean, show the evidence, and suggest how each one can be fixed.

2. Scope and Setup

2.1 Target Details

Item	Details
Target Device	OWASP IoTGoat v1.0
Target IP Address	192.168.57.128
Attacker Machine	Kali Linux 2026
Attacker IP Address	192.168.57.129
Network Type	VMware Host-Only (isolated, no internet)
Test Type	Black-box penetration test
Date of Testing	May 2026

2.2 Methodology

I followed the standard five-phase penetration testing process that is used in the industry. This helped me stay organised and make sure I did not miss anything important.

Phase	What I Did
1 – Reconnaissance	Used Nmap to scan the network and find what ports and services were running on the device
2 – Enumeration	Looked more closely at each service to understand what version it was and how it was configured
3 – Vulnerability Analysis	Identified which services had known weaknesses or poor configurations

4 – Exploitation	Tested the vulnerabilities to confirm they were real and show the impact
5 – Reporting	Wrote up all the findings with evidence and recommendations

2.3 Tools Used

Tool	Purpose
Nmap 7.95	Scanned the network, found open ports and identified what services were running
Nikto v2.5.0	Scanned the web interface for missing security headers and other issues
Netcat (nc)	Connected to the backdoor service and accessed the device shell
John the Ripper	Attempted to crack the password hashes extracted from the device
Telnet client	Tested the Telnet service for unauthorised access
Wireshark	Captured network traffic to check for unencrypted data

3. Summary of Findings

The table below gives a quick overview of all the vulnerabilities I found during the assessment. I found 14 issues in total. Four of them are critical, which means they could give an attacker complete control of the device.

ID	Finding	Severity	Status
F-001	Backdoor with no password – gives root access on port 5515	Critical	Confirmed & Exploited
F-002	Telnet is running – sends everything in plain text	Critical	Confirmed
F-003	Password hashes stored using weak MD5 and readable by anyone	Critical	Confirmed & Exploited
F-004	WiFi has no encryption at all	Critical	Confirmed
F-005	Linux kernel from 2019 – over 7 years out of date	High	Confirmed
F-006	dnsmasq version 2.73 – old version with known vulnerabilities	High	Confirmed
F-007	Backup config files left on the device and readable	High	Confirmed
F-008	Missing X-Frame-Options header – clickjacking risk	High	Confirmed
F-009	No HSTS header – connection can be downgraded to HTTP	High	Confirmed
F-010	SSL certificate is self-signed, not trusted	Medium	Confirmed
F-011	Certificate name does not match the device IP address	Medium	Confirmed
F-012	Missing X-Content-Type-Options header	Medium	Confirmed
F-013	ETag header leaks filesystem information (CVE-2003-1418)	Medium	Confirmed
F-014	Duplicate entry in the /etc/shadow file	Low	Observed

4. Detailed Findings

Each finding below explains what I discovered, how I found it, what the risk is, and how it can be fixed.

F-001 – Backdoor Service on Port 5515 (No Authentication)

Finding ID	F-001
Title	Unauthenticated Backdoor Giving Root Shell Access on Port 5515
Severity	Critical
Port / Service	5515/TCP
Tool Used	Nmap, Netcat
What I Found	When I ran my Nmap scan I noticed that port 5515 had a very unusual banner – it said 'Successfully Connected to IoTGoat Backdoor'. I connected to it using Netcat and immediately got a shell on the device. Running the 'id' command showed I was running as root (uid=0), which is the highest level of access you can have on a Linux system. No username or password was needed at any point.
Why It Is Risky	This is the most serious finding in the report. Anyone on the same network as the device can connect to this port and take full control of the device instantly. From that position they could read all files, change settings, steal credentials, install malware, or use the device to attack other systems on the network.
Evidence	Nmap banner on port 5515: '***]Successfully Connected to IoTGoat Backdoor[***]'. Netcat connection successful. Command 'id' returned: uid=0(root) gid=0(root).
How to Fix It	The backdoor service must be removed immediately. All running processes and open ports should be audited. The device firmware should be replaced with a clean, verified version. Network firewall rules should block access to unnecessary ports.

Commands I ran:

```
nc 192.168.57.128 5515
id
Output: uid=0(root) gid=0(root)
```

F-002 – Telnet Running on Port 65534

Finding ID	F-002
Title	Telnet Service Active on Port 65534 – Unencrypted Protocol
Severity	Critical
Port / Service	65534/TCP – BusyBox telnetd

Tool Used	Nmap, Telnet client
What I Found	Nmap found a Telnet service running on port 65534. Telnet is an old protocol that does not encrypt anything it sends over the network. I connected to it using the Telnet client and was presented with a login prompt for the IoTGoat device. While I could not log in with the default passwords I tried, the service is still active and accepting connections.
Why It Is Risky	Telnet sends the username, password, and everything typed in the session as plain text over the network. Anyone who is monitoring network traffic with a tool like Wireshark can see all of this in real time. If an attacker captures valid credentials during a legitimate admin session, they can use those to log in themselves.
Evidence	Nmap output: 65534/tcp open telnet BusyBox telnetd. Telnet connection established and login prompt confirmed.
How to Fix It	Disable Telnet completely. Use SSH for all remote access instead. Configure SSH to use key-based authentication and disable password login. Add a firewall rule to block port 65534 from outside the management network.

F-003 – Weak Password Hashes in /etc/shadow

Finding ID	F-003
Title	Password Hashes Using Old MD5 Algorithm – Readable Without Restriction
Severity	Critical
Port / Service	N/A – File system access via F-001
Tool Used	Netcat (backdoor shell), John the Ripper
What I Found	After getting root access through the backdoor, I read the /etc/shadow file which contains the password hashes for all user accounts. I found hashes for both the root account and iotgoatuser. The hashes start with '\$1\$' which tells us they are using the MD5 algorithm. MD5 has been considered broken for many years and should not be used for passwords. I ran the hashes through John the Ripper using the rockyou wordlist.
Why It Is Risky	Even though I did not crack the passwords in the time available, the fact that the hashes use MD5 means they are much weaker than they should be. With modern GPU-based cracking tools, MD5 hashes can be tested at billions of guesses per second. In a real attack scenario, these hashes would likely be cracked quickly. Once cracked, the passwords could be used to log in via SSH or the web panel.
Evidence	File /etc/shadow read successfully via backdoor shell. Hashes confirmed: root:\$1\$J17H1VOG\$... and iotgoatuser:\$1\$79bz0K8z\$... – both using MD5-crypt (\$1\$) format.
How to Fix It	Switch to a stronger hashing algorithm such as SHA-512 (which starts with '\$6\$'). This is the default on most modern Linux systems. Use long, complex passwords for all accounts. Remove any accounts that are not needed. Make sure /etc/shadow can only be read by root.

Hashes extracted:

```
root:$1$Jl7H1VOG$Wgw2F/C.nLNTC.4pwDa4H1:18145:0:99999:7:::  
iotgoatuser:$1$79bz0K8z$Ii6Q/if83F1QodGmkb4Ah.:18145:0:99999:7:::
```

F-004 – WiFi Has No Encryption

Finding ID	F-004
Title	Wireless Access Point Broadcasting with No Encryption Enabled
Severity	Critical
Port / Service	Wireless – wl0 (802.11a)
Tool Used	Netcat, /etc/config/wireless file
What I Found	After reading the wireless configuration file from the device, I found that the WiFi access point is set up with encryption turned off completely. The SSID being broadcast is 'OWASP IoTGoat' and any device nearby can connect to it with no password.
Why It Is Risky	With no WiFi encryption, anyone within range of the device can join the network freely. Once on the network, they can access all the services running on the device, including the backdoor found in F-001. All wireless traffic is also visible to anyone monitoring the airwaves nearby, meaning any data the device transmits can be intercepted.
Evidence	Configuration file /etc/config/wireless contained: option encryption 'none' and option ssid 'OWASP IoTGoat'. No authentication required to associate with the access point.
How to Fix It	Enable WPA3 encryption on the wireless interface. If WPA3 is not supported, use WPA2 with AES as a minimum. Set a strong passphrase. Consider disabling the SSID broadcast if the network does not need to be publicly visible.

F-005 – Linux Kernel from January 2019

Finding ID	F-005
Title	Device Running Linux Kernel 4.14.95 – Compiled January 2019
Severity	High
Port / Service	N/A – Operating system level
Tool Used	Netcat (uname -a via shell)
What I Found	Running 'uname -a' on the device showed the kernel version is 4.14.95, which was compiled on 30 January 2019. This is over seven years old. The Linux kernel receives regular security updates and a kernel this old has many known vulnerabilities that have never been patched on this device.

Why It Is Risky	Old kernels are one of the most common attack paths used in IoT exploitation. Attackers can look up published vulnerabilities for kernel 4.14.95 and use them to escalate privileges or bypass security controls. In many real-world IoT attacks, the kernel version alone gives attackers the information they need to select a working exploit.
Evidence	Command 'uname -a' output: Linux IoTGoat 4.14.95 #0 SMP Wed Jan 30 12:21:02 2019 i686 GNU/Linux
How to Fix It	Update the device firmware to the latest stable version. This will include a newer, patched kernel. Set up a regular firmware review schedule. Subscribe to security advisories from the firmware vendor.

F-006 – Outdated dnsmasq Version 2.73

Finding ID	F-006
Title	DNS Service Running dnsmasq 2.73 – Multiple Known CVEs
Severity	High
Port / Service	53/TCP – DNS
Tool Used	Nmap (dns-nsid script)
What I Found	Nmap identified that the DNS service is running dnsmasq version 2.73. This is a significantly outdated version that has several publicly known vulnerabilities, including ones that allow DNS cache poisoning and in some cases remote code execution. The version number is also being disclosed in the DNS response banner, which makes it easy for an attacker to identify the exact version and look up matching exploits.
Why It Is Risky	An attacker who exploits dnsmasq vulnerabilities could poison the DNS cache, redirecting the device and any clients using it to malicious websites. This could lead to credential theft or malware installation. Version disclosure also makes targeted attacks easier.
Evidence	Nmap script dns-nsid output: bind.version: dnsmasq-2.73. Port 53 confirmed open.
How to Fix It	Update dnsmasq to the latest version. Disable the version banner in the dnsmasq configuration file. Restrict the DNS service to only listen on internal interfaces. Set up monitoring to alert on unusual DNS activity.

F-007 – Backup Config Files Left on the Device

Finding ID	F-007
Title	Network and DNS Backup Files Stored in Accessible Locations on Filesystem
Severity	High
Port / Service	N/A – File system access via F-001

Tool Used	Netcat, cat command
What I Found	While reading the dnsmasq_setup.sh script found in the root directory, I noticed it references backup copies of configuration files: /etc/config/network.bak and /etc/dnsmasq.conf.bak. These files were left on the device and can be read by anyone with access to the shell. They contain network configuration details that could help an attacker understand how the wider network is set up.
Why It Is Risky	Configuration backups can reveal network topology, DNS settings, and sometimes credentials that were valid at the time the backup was made. This kind of information is very useful to an attacker during the reconnaissance phase of a wider attack. Leaving unnecessary files on a production device also increases the overall attack surface.
Evidence	File /dnsmasq_setup.sh read via shell. References: netbak=/etc/config/network.bak and dnsbak=/etc/dnsmasq.conf.bak. Both paths accessible on the live filesystem.
How to Fix It	Remove backup files from deployed devices. If backups are needed, store them securely off the device in an encrypted location. Review all scripts on the device and remove any references to sensitive file paths. Implement a policy that production IoT devices should only contain files needed for normal operation.

F-008 – X-Frame-Options Header Missing

Finding ID	F-008
Title	Web Admin Panel Does Not Send X-Frame-Options Header
Severity	High
Port / Service	443/TCP – HTTPS web interface
Tool Used	Nikto v2.5.0
What I Found	When I scanned the web admin panel with Nikto, it flagged that the X-Frame-Options header is missing from the HTTP responses. This header tells browsers whether the page is allowed to be loaded inside an iframe on another website.
Why It Is Risky	Without this header, an attacker can embed the admin login page inside a hidden iframe on a fake website. If an administrator visits that fake website while already logged in, their clicks could be secretly redirected to perform actions on the admin panel without them realising. This type of attack is called clickjacking.
Evidence	Nikto scan result: 'The anti-clickjacking X-Frame-Options header is not present.'
How to Fix It	Add the header 'X-Frame-Options: DENY' to all responses from the web server. This can be configured in the uhttpd settings on OpenWrt. It is a simple change that takes very little effort but removes the clickjacking risk entirely.

F-009 – HSTS Header Not Set

Finding ID	F-009
Title	Strict-Transport-Security Header Missing – SSL Stripping Risk
Severity	High
Port / Service	443/TCP – HTTPS web interface
Tool Used	Nikto v2.5.0
What I Found	Nikto found that the HTTPS web interface does not send the Strict-Transport-Security (HSTS) header. HSTS is a security header that tells browsers to always use HTTPS and never fall back to plain HTTP when connecting to this server.
Why It Is Risky	Without HSTS, a man-in-the-middle attacker can intercept the first connection before the HTTPS redirect happens and strip the encryption layer. This is called an SSL stripping attack. The victim ends up on a plain HTTP version of the site without realising it, and everything they type including passwords is sent in plain text.
Evidence	Nikto scan result: 'The site uses TLS and the Strict-Transport-Security HTTP header is not defined.'
How to Fix It	Add the header 'Strict-Transport-Security: max-age=31536000; includeSubDomains' to all HTTPS responses. Configure the web server to reject plain HTTP connections and redirect everything to HTTPS from the start.

F-010 – Self-Signed SSL Certificate

Finding ID	F-010
Title	HTTPS Using a Self-Signed Certificate Not From a Trusted Authority
Severity	Medium
Port / Service	443/TCP – HTTPS
Tool Used	Nmap ssl-cert script
What I Found	The Nmap scan showed that the HTTPS service is using a self-signed certificate. The certificate was issued by the device itself (OpenWrt2e410863) rather than a recognised Certificate Authority. The details in the certificate also used placeholder values like countryName=ZZ and Somewhere for the location, showing it was auto-generated without any real identity verification.
Why It Is Risky	Self-signed certificates cause browsers to show security warnings, which can train users to click through warnings without thinking. They also provide no real proof of the server's identity, which makes it easier for attackers to run man-in-the-middle attacks using their own fake certificate without the victim noticing much difference.
Evidence	Nmap ssl-cert output: Subject contains commonName=OpenWrt, organizationName=OpenWrt2e410863, countryName=ZZ. Issuer is the same as Subject, confirming it is self-signed.
How to Fix It	Replace the self-signed certificate with one issued by a trusted Certificate Authority. For internal use, set up an internal CA and push the root certificate to all devices and

browsers in the organisation. This allows proper certificate validation without public CA costs.

F-011 – Certificate Hostname Mismatch

Finding ID	F-011
Title	SSL Certificate Name Does Not Match the Device IP Address
Severity	Medium
Port / Service	443/TCP – HTTPS
Tool Used	Nikto v2.5.0
What I Found	Nikto flagged that the certificate's Common Name is 'OpenWrt' but the device is being accessed by its IP address 192.168.57.128. These do not match, which is a certificate validation failure.
Why It Is Risky	Certificate mismatches are a sign that proper certificate management is not in place. In a real attack, this mismatch makes it slightly easier for an attacker to pass off a fake certificate because users are already used to seeing warnings about this device's certificate.
Evidence	Nikto output: 'Hostname 192.168.57.128 does not match certificate names: OpenWrt. See CWE-297.'
How to Fix It	When issuing certificates for IoT devices, include the device IP address or hostname in the Subject Alternative Name field of the certificate. This ensures the certificate matches how the device is actually accessed on the network.

F-012 – Missing X-Content-Type-Options Header

Finding ID	F-012
Title	X-Content-Type-Options Header Not Present on Web Interface
Severity	Medium
Port / Service	443/TCP – HTTPS
Tool Used	Nikto v2.5.0
What I Found	Nikto reported that the web server is not sending the X-Content-Type-Options header. This header stops browsers from guessing what type of file a response is and forces them to use the declared content type.
Why It Is Risky	Without this header, a browser might interpret a file with an incorrect content type differently from how it was intended. For example, a text file could be interpreted as JavaScript and executed. This is called MIME type sniffing and it can be used to run malicious scripts.

Evidence	Nikto output: 'The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type.'
How to Fix It	Add the header 'X-Content-Type-Options: nosniff' to all HTTP responses. This is a single line configuration change in the web server settings and requires minimal effort.

F-013 – ETag Header Leaking Inode Info (CVE-2003-1418)

Finding ID	F-013
Title	HTTP ETag Header Exposing Internal Filesystem Inode Numbers
Severity	Medium
Port / Service	443/TCP – HTTPS
Tool Used	Nikto v2.5.0
What I Found	Nikto detected that the web server is sending ETag headers that include the filesystem inode number of the file being served. The specific values found were: inode 408, size 495, and a modification date of January 30 2019. CVE-2003-1418 is a known vulnerability related to this behaviour in Apache-style servers.
Why It Is Risky	Inode numbers reveal information about the internal structure of the server's filesystem. The modification date also confirms that the firmware has not been updated since January 2019, which is useful information for an attacker trying to select known exploits for that firmware version.
Evidence	Nikto output: 'Server may leak inodes via ETags, inode: 408, size: 495, mtime: Wed Jan 30 07:21:02 2019. See CVE-2003-1418.'
How to Fix It	Configure the web server to generate ETags without including inode information. In most web servers this can be done by changing the ETag policy to use only file size and modification time. Alternatively, ETags can be disabled and Cache-Control headers used instead.

F-014 – Duplicate Entry in /etc/shadow

Finding ID	F-014
Title	The dnsmasq Account Appears Twice in the /etc/shadow File
Severity	Low
Port / Service	N/A – File system access via F-001
Tool Used	Netcat, cat command
What I Found	When I read the /etc/shadow file through the backdoor shell, I noticed that the dnsmasq account appears twice on consecutive lines with identical content. This is a misconfiguration that suggests the file has been edited manually at some point without proper checking.

Why It Is Risky	While a duplicate entry like this is not directly exploitable on its own, it indicates poor configuration management. It could also potentially cause unexpected behaviour with certain system tools that process the shadow file. More importantly, it suggests that other parts of the configuration may have been modified in an ad hoc and untracked way, which increases the chances of other misconfigurations existing.
Evidence	File /etc/shadow read via shell. Two identical consecutive lines found: dnsmasq:x:0:0:99999:7:::
How to Fix It	Remove the duplicate dnsmasq line from /etc/shadow. Review the rest of the file and other system files for similar issues. Going forward, use proper configuration management tools rather than manual file editing to ensure consistency and audit trails.

5. Risk Assessment

The table below shows all findings rated by how likely they are to be exploited and how much damage they could cause. Critical findings combine both high likelihood and high impact.

Finding ID	Title	Likelihood	Impact	Overall Risk
F-001	Backdoor on port 5515	High	High	Critical
F-002	Telnet service exposed	High	High	Critical
F-003	Weak MD5 password hashes	High	High	Critical
F-004	WiFi with no encryption	High	High	Critical
F-005	Outdated Linux kernel 2019	Medium	High	High
F-006	Old dnsmasq 2.73	Medium	High	High
F-007	Backup config files exposed	High	Medium	High
F-008	Missing X-Frame-Options	Medium	Medium	High
F-009	No HSTS header	Medium	Medium	High
F-010	Self-signed certificate	Low	Medium	Medium
F-011	Certificate hostname mismatch	Low	Medium	Medium
F-012	Missing X-Content-Type-Options	Medium	Low	Medium
F-013	ETag inode leakage	Medium	Low	Medium
F-014	Duplicate shadow entry	Low	Low	Low

6. Recommendations

The table below lists what should be done to fix each finding. The ones marked as immediate need to be done before the device is used anywhere near a real network.

ID	Action Needed	Priority
F-001	Remove the backdoor service on port 5515 and replace the firmware with a clean version	Immediate
F-002	Turn off Telnet and switch to SSH with key-based authentication only	Immediate
F-003	Change password hashing to SHA-512 and enforce strong password requirements	Immediate
F-004	Turn on WPA3 encryption on the wireless access point	Immediate
F-005	Update the device firmware to get a newer, patched kernel	Short-term
F-006	Update dnsmasq and disable the version banner in the config	Short-term
F-007	Delete backup config files from the device filesystem	Short-term
F-008	Add X-Frame-Options: DENY header to the web server config	Short-term
F-009	Add the Strict-Transport-Security header to HTTPS responses	Short-term
F-010	Get a proper certificate from a trusted Certificate Authority	Medium-term
F-011	Make sure the certificate includes the correct hostname or IP in the SAN field	Medium-term
F-012	Add X-Content-Type-Options: nosniff to web server responses	Medium-term
F-013	Configure ETag headers to not include inode numbers	Medium-term
F-014	Remove the duplicate dnsmasq line from /etc/shadow	Low-priority

7. Conclusion

This assessment tested the security of an IoT device running OWASP IoTGoat firmware. Overall, the device is in a very poor security state and would not be safe to deploy in any real organisation in its current form.

The biggest problem found was an open backdoor on port 5515 that gave immediate root-level access to the device with no password needed. On top of that, the WiFi had no encryption, Telnet was running in plain text, and the password hashes were stored in a weak and outdated format. Any one of these issues on its own would be serious, but having all four together makes the device extremely vulnerable.

A lot of the other issues, like the 2019 kernel and old dnsmasq version, come down to the fact that the firmware was never updated. This is a common problem with IoT devices in the real world because once they are installed and working, people tend to leave them alone. But outdated software means outdated security.

To summarise, the four critical issues need to be fixed straight away, and the remaining issues should be worked through over the following weeks. Going forward, any IoT devices used by an organisation should go through a security review before deployment, be placed on a separate network segment away from critical systems, and have a firmware update plan in place.

8. Appendices

Appendix A – Nmap Scan Results

Command used:

```
nmap -sV -sC -p- 192.168.57.128 -oN recon_scan.txt
```

Open ports found:

```
22/tcp    open  ssh      Dropbear sshd (protocol 2.0)
53/tcp    open  domain   dnsmasq 2.73
80/tcp    open  http     LuCI Lua http config
443/tcp   open  ssl/http LuCI Lua http config
5515/tcp  open  unknown  [***]Successfully Connected to IoTGoat's
Backdoor[***]
65534/tcp open  telnet   BusyBox telnetd
```

Appendix B – System Details from Backdoor Shell

```
id          => uid=0(root) gid=0(root)
uname -a => Linux IoTGoat 4.14.95 #0 SMP Wed Jan 30 12:21:02 2019
i686 GNU/Linux
```

Appendix C – /etc/passwd File Contents

```
root:x:0:0:root:/root:/bin/ash
daemon:*:1:1:daemon:/var:/bin/false
ftp:*:55:55:ftp:/home/ftp:/bin/false
network:*:101:101:network:/var:/bin/false
nobody:*:65534:65534:nobody:/var:/bin/false
dnsmasq:x:453:453:dnsmasq:/var/run/dnsmasq:/bin/false
iotgoatuser:x:1000:1000::/root:/bin/ash
```

Appendix D – Password Hashes from /etc/shadow

```
root:$1$Jl7H1VOG$Wgw2F/C.nLNTC.4pwDa4H1:18145:0:99999:7:::
iotgoatuser:$1$79bz0K8z$Ii6Q/if83F1QodGmkb4Ah.:18145:0:99999:7:::
```

Hash type: \$1\$ = MD5-crypt (weak, deprecated)

Appendix E – Wireless Configuration

```
option encryption 'none'
option ssid 'OWASP IoTGoat'
option hwmode '11a'
option mode 'ap'
```

Appendix F – Nikto Web Scan Results

Command used:

```
nikto -h https://192.168.57.128 -ssl -o nikto_output.txt
```

Key results:

- + Server may leak inodes via ETags (CVE-2003-1418)
- + X-Frame-Options header not present
- + Strict-Transport-Security header not defined
- + X-Content-Type-Options header not set
- + Hostname does not match certificate name: OpenWrt

Appendix G – Disclaimer

All testing in this report was carried out on OWASP IoTGoat v1.0, which is a deliberately vulnerable device created for training and educational purposes. The entire assessment was done inside an isolated VMware Host-Only network with no connection to the internet or any real systems. This report has been written as part of an academic Continuous Assessment and is not intended for use outside of an educational context.